NAGFANI FOODS ERP (V1.0 CLASSIC)

COMPLETE FUNCTIONAL SPECIFICATION, STATE MACHINES & API SYSTEM MAP

This document serves as the canonical technical map and functional blueprint of the Nagfani Foods ERP (Version 1.0 Classic). Engineered specifically for machine-readability, AI context-ingestion, and developer auditability, it comprehensively maps all 24 frontend screens, 45+ backend API endpoints, 22 database collections, real-time WebSocket events, and the on-premises TallyPrime port 9000 XML gateway.

PLATFORM Nagfani Foods ERP	**TARGET RELEASE** V1.0 Classic Core	**DEPLOYMENT** Live Cloud & Desktop	**DATABASE** MongoDB (Mongoose)
FRONTEND React 18 + Vite PWA	**BACKEND** Node 20 + Express REST	**REAL-TIME** Socket.IO WebSocket	**ACCOUNTING** TallyPrime (Port 9000)

🤖 **AI Context Note:** This document contains structured data dictionaries, exact route parameters, JSON schemas, state machine lifecycle rules, and mathematical invariants. Text, table columns, and code signatures are formatted for direct parsing by Large Language Models, code generation engines, and automated test runners.

1. AI Manifest & System Constants

Constant Key	System Value	Architectural Significance
`VERSION_CODE`	1.0.0-classic	Identifies the baseline stable release containing classic layout and workflows.
`MONOREPO_ROOT`	`/home/rushal/Desktop/TEST SITES/Nagmani`	Monorepo containing `web/`, `api/`, `connector/`, and `installer/`.
`DEFAULT_PORT_API`	5000	Node.js Express REST API server and Socket.IO real-time hub.
`DEFAULT_PORT_WEB`	5173	Vite React Single Page Application (HashRouter based).
`DEFAULT_PORT_TALLY`	9000	TallyPrime HTTP XML server running on client Windows workstation.
`CORE_STOCK_INVARIANT`	`Physical - Reserved = Available`	Fundamental law of inventory; stock is never mutated directly without a movement record.
`TAX_JURISDICTION`	State Code `08` (Rajasthan)	Intra-state: CGST (2.5%) + SGST (2.5%) = 5%; Inter-state: IGST (5%).
`AUTH_SCHEME`	Bearer JWT (12h expiration)	Standard Authorization header: `Authorization: Bearer <TOKEN>`.
`CONNECTOR_AUTH`	Header `x-api-key`	Dedicated long-lived API key authenticating the Windows Tally gateway.

2. Executive System Architecture & Topology



2.1 Monorepo Structure & Module Boundaries

- `web/`: Vite + React 18 single page application. HashRouter navigation, Tailwind CSS styling, TanStack Query for cache invalidation, Zustand for auth state.
- `api/`: Express application in ES modules. Mongoose models, controllers, Zod/Joi validation, Socket.IO broadcast handlers, centralized transaction wrappers.
- `connector/` & `tally-connector-client/`: Python 3.10+ gateway application, Tkinter desktop GUI, `tally_xml.py` XML builders and parsers, `pull_tally_masters.py` importer.
- `installer/`: NSIS installer compiler scripts (`nagfani_installer.nsi`) producing self-extracting `Setup.exe` with embedded Python runtime and offline wheel dependencies.

2.2 UI Design System Tokens (V1 Classic)

Token Name	Color Hex	RGB / Class	Usage Context
Brand Primary	`#D9580B`	Heritage Orange	Primary action buttons, brand badges, active tab underlines, price accents.
Brand Secondary	`#03353A`	Deep Mewar Teal	Top header navigation bar, main sidebar background, table header text.
Brand Dark Teal	`#02282C`	Dark Teal Shade	Active sidebar item hover state, header logo box borders.
Background	`#F8FAFC`	Slate 50	Page body background, muted card borders.
Card Surface	`#FFFFFF`	Pure White	Content containers, modal dialogs, data table backgrounds.
Status Success	`#10B981`	Emerald 500	"Delivered" status, "Synced" Tally status, positive stock levels.
Status Warning	`#F59E0B`	Amber 500	"Packed" status, "Pending" outbox queue, low-stock threshold warning.
Status Danger	`#EF4444`	Red 500	"Cancelled" status, "Failed" sync attempts, stock shortfall alerts.

3. Core Business Logic Engines & Mathematical Foundations

3.1 Multi-Godown Stock Engine

> **CRITICAL SYSTEM RULE:** Stock quantities in `stock_levels` are NEVER directly overwritten or updated via direct arithmetic. Every inventory change MUST be written as an immutable document in `stock_movements` wrapped inside a MongoDB session transaction.

The state of inventory for any item in any godown is governed by the following mathematical identity:

```
Available Stock = Physical Stock - Reserved Stock
```

Movement Type	Physical Effect	Reserved Effect	Available Effect	Triggering Business Event
`purchase`	`+ qty`	0	`+ qty`	Supplier purchase invoice receipt confirmed.
`production_in`	`+ qty`	0	`+ qty`	Manufacturing batch completed & packed.
`consumption`	`- qty`	0	`- qty`	Raw material consumed in recipe batch.
`reserve`	0	`+ qty`	`- qty`	Customer sales order booked in "received" status.
`release`	0	`- qty`	`+ qty`	Order cancelled or line items deleted.
`pack`	0	0	0	Warehouse packs goods into delivery staging bay.
`ship`	`- qty`	`- qty`	0	Delivery van departs warehouse with consignment.
`transfer_out`	`- qty`	0	`- qty`	Source godown in godown-to-godown transfer.
`transfer_in`	`+ qty`	0	`+ qty`	Destination godown in godown-to-godown transfer.
`adjustment`	`± qty`	0	`± qty`	Audit adjustment (physical count recount).
`wastage`	`- qty`	0	`- qty`	Damaged goods, expired packets, frying residue.

3.2 Order Lifecycle & State Machine

```
[Draft / New Order Entry]
          |
          ▼
    ( 1. RECEIVED ) ──[Order Booked]──> Reserves inventory (Reserved +qty, Available -qty)
          |
          ▼
    ( 2. PACKED )   ──[Staged at Bay]─> Physical & Reserved stay intact; order locked from user edit
          |
          ▼
    ( 3. SHIPPED )  ──[Van Departs]───> Physical stock deducted (Physical -qty, Reserved -qty)
          |
          ▼
    ( 4. DELIVERED )──[Customer Receives / COD Collected]──> Triggers Invoice & Payment entry
          |
          ▼
    [ 5. INVOICE ISSUED & TALLY SYNCED ]

* CANCEL GUARD: Order can be cancelled ONLY from 'RECEIVED' or 'PACKED'. Once 'SHIPPED', cancel is blocked.
```

3.3 Customer Rate-Memory Algorithm

When an order line item is added, the system automatically suggests the pricing for that specific customer using a strict fallback hierarchy:

1. **Customer Historical Rate:** Queries `rate_history` for `partyId` + `itemId` sorted by `date: -1`. If found, returns the most recent agreed rate.
2. **Item Standard MRP:** If no prior transaction exists for that customer, defaults to the item's `mrp` from `items`.
3. **Item Standard Cost:** If MRP is undefined, falls back to `standardCost` or base Rs 100.

3.4 Recipe BOM Batch Scaling & Manufacturing Variance

Recipes are defined for a standard base batch (typically 100 kg or 100 packs). During production entry, quantities are dynamically scaled:

```
Theoretical Line Qty = BOM.baseLineQty * (ActualBatchProducedQty / BOM.baseQty)
Variance = ActualConsumptionQty - TheoreticalLineQty
```

Accounting & Inventory Impact: In a single atomic transaction:

- Each raw material item has a `consumption` movement logged (godown physical stock decreases).
- The finished goods item has a `production_in` movement logged (godown physical stock increases).
- If `wastage > 0`, a `wastage` movement is logged on the finished product.
- A Manufacturing Stock Journal entry is enqueued to `sync_outbox` for TallyPrime posting.

3.5 Customer Running Balance Ledger & Payment Mechanics

Customer accounts use an open-item running balance calculation:

```
Invoice Generated (Debit)  ──> Balance increases (+ Grand Total)
Payment Recorded (Credit)  ──> Balance decreases (- Paid Amount)
Net Ledger Balance         ──> Current Receivable (Outstanding Debt)
```

For suppliers, the polarity is inverted: purchase bills credit the supplier account (increasing accounts payable, represented as a negative balance).

4. Complete Data Dictionary & Schema Specification

All core business collections inherit timestamp fields (`createdAt` , `updatedAt`) and synchronization sub-documents where applicable.

1. User EntityCollection: users

Field Name	Data Type	Constraints	Description & Rules
`name`	String	Required, trim	Full legal name of the user (e.g. "Suresh Meena").
`username`	String	Required, unique, lowercase	Unique login identifier (alphanumeric, no spaces).
`mobile`	String	Required, trim	10-digit primary mobile contact number.
`passwordHash`	String	Required	bcrypt hashed password string (cost factor 10).
`roleId`	ObjectId	Ref: Role, Required	Links to assigned role defining permission rights.
`active`	Boolean	Default: true	Soft-deactivation flag; deactivated users cannot log in.
`mustChangePassword`	Boolean	Default: false	Forces password change modal on first login.
`lastLogin`	Date	Nullable	Timestamp of last authenticated session.

2. Role EntityCollection: roles

Field Name	Data Type	Constraints	Description & Rules
`name`	String	Required, trim	Display name of role (e.g. "Order Executive").
`code`	String	Required, uppercase, unique	Role code: `ADMIN` , `ORDER` , `PACKING` , `DELIVERY` , `ACCOUNTS` .
`description`	String	Default: ''	Detailed scope of operational duties.
`permissions`	[String]	Array of strings	Permission keys (e.g. `['order.create', 'order.view']` or `['*']`).
`active`	Boolean	Default: true	Status flag for role availability.

3. Item Entity (Raw & Finished Goods)Collection: items

Field Name	Data Type	Constraints	Description & Rules
`name`	String	Required, trim	Commercial product name (e.g. "Urad Hari Mirch Papad").
`sku`	String	Required, unique, uppercase	Unique inventory SKU code (e.g. "URAD-HARI-MIRCH-500G").
`category`	String	Enum: raw, finished, packaging, other	Classification determining BOM consumption vs sales eligibility.
`productGroup`	String	Default: ''	Grouping category: PAPAD, KHAKHRA, NAMKEEN, AACHAR, MASALA.
`packWeightGrams`	Number	Default: 0	Gram weight per unit pack (e.g. 200, 500).
`unitId`	ObjectId	Ref: Unit, Required	Base measurement unit (e.g. kg, pcs).
`mrp`	Number	Default: 0	Maximum Retail Price printed on consumer packaging.
`standardCost`	Number	Default: 0	Standard purchase or production cost per unit.
`gstRate`	Number	Default: 5	GST tax rate percentage (0, 5, 12, 18).
`hsn`	String	Default: '1905'	Harmonized System Nomenclature code for invoicing.
`minStock`	Number	Default: 10	Low-stock alert reorder threshold.
`tallyGuid`	String	Default: null	Remote GUID mapping record to Tally StockItem master.
`active`	Boolean	Default: true	Master availability status.

4. Party Entity (Customers & Suppliers)Collection: parties

Field Name	Data Type	Constraints	Description & Rules
`type`	String	Enum: customer, supplier	Master entity type.
`segment`	String	Enum: b2b, b2c	Classification driving invoice numbering & Challan generation.
`b2bType`	String	Enum: wholesale, retailer, distributor, other	Sub-category for wholesale discount tiers.
`name`	String	Required, trim	Business firm name or individual customer name.
`mobile`	String	Required, index	Primary 10-digit WhatsApp/calling contact.
`address` / `city` / `pincode`	String	Required	Physical billing and delivery address.
`areaId`	ObjectId	Ref: Area, Required	Logistics beat area (one of 31 Udaipur beats).
`mapsUrl`	String	Default: ''	Direct Google Maps navigation URL for driver routing.
`gstin`	String	Uppercase, trim	15-character GST identification number.
`balance`	Number	Default: 0	Running ledger balance (Positive = Due from customer, Negative = Payable to vendor).
`creditLimit` / `creditDays`	Number	Default: 0	Credit risk controls enforcing max credit and payment term days.
`tallyGuid`	String	Default: null	Remote GUID mapping record to Tally Sundry Debtor/Creditor ledger.

5. Order EntityCollection: orders

Field Name	Data Type	Constraints	Description & Rules
`orderNo`	String	Required, unique	Sequential identifier formatted as `ORD-000123`.
`partyId`	ObjectId	Ref: Party, Required	Customer account booking the consignment.
`segment`	String	Enum: b2b, b2c	Inherited from party at booking time.
`areaId`	ObjectId	Ref: Area, Required	Delivery area beat for routing and run sheets.
`godownId`	ObjectId	Ref: Godown, Required	Warehouse facility fulfilling and reserving inventory.
`items`	Array of sub-docs	Required, non-empty	Array containing `itemId`, `sku`, `name`, `qty`, `rate`, `discount`, `taxRate`, `lineTotal`.
`subtotal` / `taxTotal` / `grandTotal`	Number	Required	Aggregated monetary totals in INR.
`challanNo`	String	Default: ''	Delivery Challan number (mandatory for B2C, e.g. `DC-000125`).
`invoiceId`	ObjectId	Ref: Invoice, Nullable	Links to generated Tax Invoice document.
`status`	String	Enum: received, packed, shipped, delivered, cancelled	Operational lifecycle stage of the order.
`paymentStatus`	String	Enum: unpaid, partial, paid	Settlement status against order total.
`paidAmount`	Number	Default: 0	Cumulative sum of payments credited to this order.

6. Bill of Materials (BOM) EntityCollection: boms

Field Name	Data Type	Constraints	Description & Rules
`productId`	ObjectId	Ref: Item, Required	Finished food item produced by this recipe.
`name`	String	Required, trim	Recipe title (e.g. "Standard Urad Papad Recipe").
`version`	Number	Required, default: 1	Sequential revision number; higher version takes precedence.
`baseQty`	Number	Required, default: 100	Standard batch yield quantity (e.g. 100 kg or 100 packs).
`lines`	Array of sub-docs	Required	List of raw materials with `itemId`, `sku`, `name`, `qty`, `unitSymbol`.
`active`	Boolean	Default: true	Only active BOMs appear in production dropdowns.

7. Production Entry (Manufacturing Journal)Collection: production_entries

Field Name	Data Type	Constraints	Description & Rules
`productionNo`	String	Required, unique	Sequential identifier formatted as `PRD-000045`.
`date`	Date	Required, default: now	Date of kitchen or factory production batch.
`productId`	ObjectId	Ref: Item, Required	Finished goods item manufactured.
`qty`	Number	Required, > 0	Actual quantity of finished packs manufactured.
`godownId`	ObjectId	Ref: Godown, Required	Godown where raw materials are consumed and goods inwarded.
`bomId` / `bomVersion`	ObjectId / Number	Required	Recipe template reference used for scaling.
`batchNo`	String	Required	Lot identifier (e.g. "BATCH-261025").
`wastage`	Number	Default: 0	Quantity of defective or scrap goods produced.
`lines`	Array of sub-docs	Required	Raw materials showing `bomQty`, `actualQty`, and calculated `variance`.

8. Synchronization Outbox & Inbox EntitiesCollections: sync_outbox & sync_inbox

Field Name	Data Type	Constraints	Description & Rules
`entityType`	String	Enum: order, invoice, payment, purchase, production, party, item	Specifies the domain entity being synchronized.
`entityId`	ObjectId	Required	Database primary key of the synchronized record.
`operation`	String	Enum: create, update, delete	Database operation to mirror into TallyPrime.
`payload`	Object	Required	Full document state exported to XML generator.
`idempotencyKey`	String	Required, unique index	Prevents duplicate voucher creation on network retries.
`status`	String	Enum: pending, sent, synced, failed, conflict	Current gateway transmission state.
`attempts`	Number	Default: 0	Failure count before backoff pauses item.
`lastError`	String	Default: null	Raw error message returned by Tally HTTP server.

5. Exhaustive REST API Route & Controller Specification

All endpoints are prefixed with `/api/v1` and enforce strict JWT authentication unless marked public.

5.1 Authentication Controller (`/api/v1/auth`)

POST **`/api/v1/auth/login`**

Access: Public · Rate Limit: 10 req/min · Auth: None

Authenticates user via username, email, or 10-digit mobile number and returns signed JWT token.

```
Request:  { "username": "admin", "password": "secure_password" }
Response: { "success": true, "token": "eyJhbG...", "user": { "id": "...", "name": "Admin", "role": { "code": "ADMIN" } } }
```

GET **`/api/v1/auth/me`**

Access: Authenticated · Auth: Bearer JWT

Returns the currently authenticated user session details, role, and mustChangePassword flag.

POST **`/api/v1/auth/change-password`**

Access: Authenticated · Auth: Bearer JWT

Updates user password. Verifies current password before hashing new password with bcrypt.

5.2 Orders Controller (`/api/v1/orders`)

GET **`/api/v1/orders`**

Access: Order, Packing, Delivery, Accounts, Admin · Query: status, areaId, segment, search, fromDate, toDate

Retrieves paginated orders with populated party, area, and godown metadata.

POST **`/api/v1/orders`**

Access: Order, Admin · Permission: `order.create` · Transaction: Yes

Books new customer order. In an atomic transaction: checks stock availability, auto-generates `orderNo` (e.g. `ORD-000123`), auto-generates B2C Delivery Challan number (`DC-000125`), computes line-level GST, logs `reserve` stock movements, enqueues `sync_outbox`, and emits `order:created` via Socket.IO.

```
Request:  { "partyId": "...", "godownId": "...", "items": [{ "itemId": "...", "qty": 10, "rate": 140 }], "paymentTerms": "COD" }
Response: { "success": true, "data": { "orderNo": "ORD-000123", "status": "received", "grandTotal": 1400 } }
```

POST / **PATCH** **`/api/v1/orders/:id/status`**

Access: Role-guarded · Transaction: Yes · Sockets: `order:status`

Transitions order lifecycle status. Enforces strict transitions:

- `packed`: Marked by Packing role; verifies staging without modifying stock reserve.
- `shipped`: Marked by Delivery/Admin; invokes `shipOrderStock()` reducing physical inventory.
- `delivered`: Marked by Delivery/Admin; finalizes delivery and updates status timeline.
- `cancelled`: Blocked if already shipped; releases reserved inventory back to available stock.

POST **`/api/v1/orders/:id/payment`**

Access: Delivery, Accounts, Admin · Permission: `payment.record` · Transaction: Yes

Records cash/UPI collection against an order. Updates order `paidAmount`, adjusts customer ledger balance, and enqueues Tally Receipt voucher.

5.3 Inventory & Multi-Godown Controller (`/api/v1/stock`)

GET **`/api/v1/stock/levels`**

Access: All Authenticated · Query: godownId, category, lowStockOnly

Returns physical, reserved, and available quantities per item and godown.

POST **`/api/v1/stock/transfers`**

Access: Admin, Stock Supervisor · Permission: `stock.transfer` · Transaction: Yes

Executes godown-to-godown inventory movement. Writes paired `transfer_out` and `transfer_in` movements atomically.

POST **`/api/v1/stock/adjustments`**

Access: Admin · Permission: `stock.adjust` · Transaction: Yes

Records physical inventory count correction or wastage write-off with mandatory audit reason.

5.4 Food Manufacturing Controller (`/api/v1/manufacturing` & `/api/v1/boms`)

GET **`/api/v1/boms/for-product/:productId`**

Access: Order, Production, Admin

Fetches the active recipe BOM for a product to auto-populate theoretical ingredient consumption in production form.

POST **`/api/v1/production`**

Access: Production Lead, Admin · Permission: `manufacturing.create` · Transaction: Yes

Confirms food manufacturing batch. Decreases raw material quantities, increases finished goods, records wastage, computes consumption variance, and creates Tally Stock Journal sync record.

5.5 Billing & Invoicing Controller (`/api/v1/invoices`)

POST **`/api/v1/invoices/generate-from-order/:orderId`**

Access: Accounts, Admin · Permission: `invoice.create` · Transaction: Yes

Generates sequential Tax Invoice (`INV-000123`). Computes CGST/SGST/IGST breakdown, debits customer ledger balance, links invoice to order, and queues Tally Sales Voucher.

5.6 Tally Gateway Integration Controller (`/api/v1/sync`)

POST **`/api/v1/sync/heartbeat`**
Access: Connector · Auth: Header `x-api-key`
Called every 3 seconds by Windows desktop connector. Updates connector online status and Tally reachability flag.

GET **`/api/v1/sync/pending`**
Access: Connector · Auth: Header `x-api-key` · Query: limit
Fetches unposted outbound records from `sync_outbox` to be converted to XML and posted to TallyPrime port 9000.

POST **`/api/v1/sync/ack`**
Access: Connector · Auth: Header `x-api-key`
Acknowledges successful Tally XML posting. Updates outbox item to `synced` and records Tally GUID in sync log.

POST **`/api/v1/sync/inbound-bulk`**
Access: Connector · Auth: Header `x-api-key`
Bulk imports customer ledgers, supplier ledgers, and stock items extracted directly from TallyPrime into the ERP cloud database.

6. Frontend V1 Classic Page & Component Specification

The V1 frontend is implemented with React 18, Vite, and Tailwind CSS. The classic shell (`Layout.jsx`) provides responsive navigation with a persistent top bar, collapsible sidebar on desktop (≥1024px), and bottom tab bar on mobile.

Route Path	Page Component	Permitted Roles	Key Functions & Interactive Triggers
`/login`	`Login.jsx`	Public	Username/mobile/email input, password field with visibility toggle, error alert, remembers redirect URL.
`/`	`Dashboard.jsx`	All Roles	Real-time KPI metrics (today sales, month sales, B2B vs B2C split, order counters, cash/bank/UPI balances, low-stock count, pending Tally queue), 30-day sales trend chart, top 5 customer debts.
`/orders`	`OrderList.jsx`	All Roles	Tabbed status filter (Received, Packed, Shipped, Delivered, All), sector filter dropdown, date range picker, search bar, one-click status transition buttons, link to Order Detail.
`/orders/new`	`NewOrder.jsx`	Order, Admin	Customer picker with live credit limit display, automatic B2B/B2C detection, item picker with **last-rate auto-suggest**, stock availability warnings, payment terms selection, live GST calculation.
`/orders/:id`	`OrderDetail.jsx`	All Roles	Consignment breakdown, audit timeline (who changed status, timestamp, note), printable Delivery Challan, Tax Invoice generator button, payment collection drawer, Tally sync status chip.
`/delivery`	`DeliveryBoard.jsx`	Delivery, Packing, Admin	Area chip selector across 31 Udaipur beats, counts per status, route map navigation buttons ("Open in Google Maps"), driver run-sheet generator, bulk dispatch confirmation.
`/customers`	`CustomerList.jsx`	Order, Accounts, Admin	Filter by B2B/B2C, sector area, outstanding > 0, live search by name/mobile, customer balance chips, link to customer ledger and profile.
`/customers/:id`	`CustomerProfile.jsx`	Order, Accounts, Admin	Comprehensive party profile: outstanding receivables, credit terms, complete order history, payment history, **rate history per SKU**, running balance ledger, direct WhatsApp link.
`/stock`	`StockOverview.jsx`	All Roles	Inventory tabs (Raw Materials, Finished Goods, Packaging), godown filter, columns for Physical, Reserved, and Available stock, low-stock highlight flags, stock movement audit drawer.
`/stock/transfer`	`StockTransfer.jsx`	Stock Lead, Admin	Godown-to-godown transfer entry: source godown, destination godown, multi-item line picker with real-time available stock validation, transfer history log.
`/stock/adjustments`	`StockAdjustments.jsx`	Admin	Inventory adjustment entry: physical count adjustment or wastage write-off, reason selector, godown selector, historical adjustment audit log.
`/items`	`ItemMaster.jsx`	Admin	Catalog management: finished goods & raw materials list, category filters, unit mapping, GST rate, HSN code, minimum reorder stock level, Tally GUID link.
`/purchases`	`PurchaseList.jsx`	Accounts, Admin	Supplier bills list, date and vendor filter, payment status badges, total procurement expenditure tracking.
`/purchases/new`	`NewPurchase.jsx`	Accounts, Admin	Supplier bill entry: vendor selection, invoice number, date, receiving godown, raw material lines with rate and GST, automated stock inward movement creation.
`/manufacturing`	`ProductionList.jsx`	Production, Admin	Manufacturing journal entries table: date, product, batch lot number, quantity produced, wastage count, link to recipe variance report.
`/manufacturing/new`	`NewProduction.jsx`	Production, Admin	Batch production entry: product selection, BOM version picker, batch yield entry, **auto-calculated consumption table** (theoretical vs actual ingredient consumption), variance logging.
`/boms`	`BOMManager.jsx`	Production, Admin	Recipe manager: view, create, copy, and version recipes. Defines base batch quantity (e.g. 100 kg) and ingredient proportions.
`/invoices`	`InvoiceList.jsx`	Accounts, Admin	Tax Invoices list, search by invoice no., challan no., or customer, payment status, PDF download button.
`/invoices/:id`	`InvoicePreview.jsx`	All Roles	Printable GST-compliant Tax Invoice & Delivery Challan, company header, customer GSTIN, HSN tax table, direct WhatsApp share button, thermal/A4 print styling.
`/payments`	`PaymentRegister.jsx`	Accounts, Delivery, Admin	Cash and bank receipt register: filter by mode (Cash, UPI, Bank Transfer), daily collection summaries, link to customer ledger.
`/outstanding`	`OutstandingList.jsx`	Accounts, Order, Admin	Customer debt aging analysis table: party name, area beat, total balance due, credit limit, days overdue, quick "Record Payment" modal.
`/reports`	`ReportsHub.jsx`	Accounts, Admin	Centralized BI analytics hub: daily/monthly sales reports, area-wise revenue, product group profitability, purchase history, production output, CSV data export.
`/sync`	`SyncOverview.jsx`	Accounts, Admin	Tally gateway status dashboard: live port 9000 reachability indicator, heartbeat timestamp, outbox pending count, error audit log, retry button, **Windows Setup.exe download link**.
`/settings`	`SettingsHub.jsx`	Admin	System configuration: user management, role permission matrix editor, area code editor (31 beats), godown manager, company profile (GSTIN, address, logo).
`/search`	`Search.jsx`	All Roles	Global unified search across orders, customers, invoices, challans, products, batches, and areas with categorised result cards.

7. Role-Based Access Control (RBAC) Permission Matrix

Nagfani Foods ERP enforces fine-grained access control. The system ships with 5 standard roles mapped across 25 operational permissions:

Permission Key	Administrative Scope	ADMIN	ORDER	PACKING	DELIVERY	ACCOUNTS
`*`	Universal Superuser Access	✅	❌	❌	❌	❌
`order.create`	Book new customer sales order	✅	✅	❌	❌	❌
`order.edit`	Modify items/rates before packed	✅	✅	❌	❌	❌
`order.pack`	Advance order status to 'packed'	✅	❌	✅	❌	❌
`order.ship`	Advance order status to 'shipped'	✅	❌	❌	✅	❌
`order.deliver`	Confirm order 'delivered' to client	✅	❌	❌	✅	❌
`order.view`	View orders & consignment lists	✅	✅	✅	✅	✅
`customer.create`	Create & edit customer profiles	✅	✅	❌	❌	✅
`customer.view`	View customer directory & balances	✅	✅	❌	✅	✅
`stock.view`	View godown inventory levels	✅	✅	✅	❌	✅
`stock.transfer`	Execute godown-to-godown transfer	✅	❌	❌	❌	❌
`stock.adjust`	Perform stock count audit/adjustment	✅	❌	❌	❌	❌
`manufacturing.create`	Record batch production & BOM	✅	❌	❌	❌	❌
`manufacturing.view`	View recipes & production journals	✅	❌	❌	❌	✅
`purchase.create`	Enter raw material vendor bills	✅	❌	❌	❌	✅
`purchase.view`	View supplier bills & purchases	✅	❌	❌	❌	✅
`invoice.create`	Generate Tax Invoice from order	✅	❌	❌	❌	✅
`invoice.view`	View & print Tax Invoices / Challans	✅	✅	✅	✅	✅
`payment.record`	Record customer payment (Cash/UPI)	✅	❌	❌	✅	✅
`payment.view`	View cash/bank ledgers & balances	✅	❌	❌	❌	✅
`delivery.view`	Access area boards & run sheets	✅	❌	✅	✅	❌
`reports.view`	Access BI sales & revenue reports	✅	❌	❌	❌	✅
`sync.view`	View Tally connector status & logs	✅	❌	❌	❌	✅
`sync.manage`	Retry failed items & edit mappings	✅	❌	❌	❌	❌
`settings.manage`	Configure users, roles, and areas	✅	❌	❌	❌	❌

8. Bidirectional TallyPrime XML Protocol & Gateway Engine

> **Gateway Connectivity:** TallyPrime runs as an on-premises HTTP web server on `http://localhost:9000`. The Python Desktop Gateway (distributed as `Setup.exe`) establishes outbound HTTPS polling to the cloud ERP API, eliminating the need for static IPs, port forwarding, or firewall holes on the client PC.

8.1 Synchronization Protocol & Polling Lifecycle

1. **Heartbeat Ping (Every 3 seconds):** Gateway sends `POST /api/v1/sync/heartbeat` containing Tally port 9000 reachability and current company name.
2. **Outbound Queue Fetch:** If Tally is responsive, gateway requests `GET /api/v1/sync/pending?limit=20`.
3. **XML Transformation:** Gateway compiles the JSON domain payload into Tally-compliant XML envelopes using `tally_xml.py`.
4. **Tally Ingestion:** Gateway sends `POST http://localhost:9000` with `Content-Type: text/xml; charset=utf-8`.
5. **Response Parsing & Acknowledgment:**
   - If `<CREATED>1</CREATED>` or `<ALTERED>1</ALTERED>`: Calls `POST /api/v1/sync/ack` with Tally GUID.
   - If `<ERRORS>>0</ERRORS>`: Calls `POST /api/v1/sync/fail` with exact line error description.

8.2 Tally XML Voucher Envelopes

Sales Voucher XML Template

```
<ENVELOPE>
  <HEADER><TALLYREQUEST>Import Data</TALLYREQUEST></HEADER>
  <BODY>
    <IMPORTDATA>
      <REQUESTDESC>
        <REPORTNAME>Vouchers</REPORTNAME>
        <STATICVARIABLES><SVCURRENTCOMPANY>NAGFANI FOODS</SVCURRENTCOMPANY></STATICVARIABLES>
      </REQUESTDESC>
      <REQUESTDATA>
        <TALLYMESSAGE xmlns:UDF="TallyUDF">
          <VOUCHER VCHTYPE="Sales" ACTION="Create">
            <DATE>20261027</DATE>
            <VOUCHERNUMBER>INV-000125</VOUCHERNUMBER>
            <PARTYLEDGERNAME>Mewar Traders</PARTYLEDGERNAME>
            <REFERENCE>ORD-000125</REFERENCE>
            <NARRATION>DC-000125 / SEC-04</NARRATION>
            <ALLLEDGERENTRIES.LIST>
              <LEDGERNAME>Mewar Traders</LEDGERNAME>
              <ISDEEMEDPOSITIVE>Yes</ISDEEMEDPOSITIVE>
              <AMOUNT>-1400.00</AMOUNT>
            </ALLLEDGERENTRIES.LIST>
            <ALLLEDGERENTRIES.LIST>
              <LEDGERNAME>Sales Account</LEDGERNAME>
              <ISDEEMEDPOSITIVE>No</ISDEEMEDPOSITIVE>
              <AMOUNT>1333.33</AMOUNT>
            </ALLLEDGERENTRIES.LIST>
            <ALLLEDGERENTRIES.LIST>
              <LEDGERNAME>CGST Output</LEDGERNAME>
              <ISDEEMEDPOSITIVE>No</ISDEEMEDPOSITIVE>
              <AMOUNT>33.33</AMOUNT>
            </ALLLEDGERENTRIES.LIST>
            <ALLLEDGERENTRIES.LIST>
              <LEDGERNAME>SGST Output</LEDGERNAME>
              <ISDEEMEDPOSITIVE>No</ISDEEMEDPOSITIVE>
              <AMOUNT>33.34</AMOUNT>
            </ALLLEDGERENTRIES.LIST>
          </VOUCHER>
        </TALLYMESSAGE>
      </REQUESTDATA>
    </IMPORTDATA>
  </BODY>
</ENVELOPE>
```

Receipt Voucher XML Template

```
<ENVELOPE>
  <HEADER><TALLYREQUEST>Import Data</TALLYREQUEST></HEADER>
  <BODY>
    <IMPORTDATA>
      <REQUESTDESC>
        <REPORTNAME>Vouchers</REPORTNAME>
        <STATICVARIABLES><SVCURRENTCOMPANY>NAGFANI FOODS</SVCURRENTCOMPANY></STATICVARIAB
      </REQUESTDESC>
      <REQUESTDATA>
        <TALLYMESSAGE xmlns:UDF="TallyUDF">
          <VOUCHER VCHTYPE="Receipt" ACTION="Create">
            <DATE>20261027</DATE>
            <VOUCHERNUMBER>RCT-000089</VOUCHERNUMBER>
            <PARTYLEDGERNAME>Mewar Traders</PARTYLEDGERNAME>
            <NARRATION>Payment received via Cash / Van 01</NARRATION>
            <ALLLEDGERENTRIES.LIST>
              <LEDGERNAME>Cash</LEDGERNAME>
              <ISDEEMEDPOSITIVE>Yes</ISDEEMEDPOSITIVE>
              <AMOUNT>-1400.00</AMOUNT>
            </ALLLEDGERENTRIES.LIST>
            <ALLLEDGERENTRIES.LIST>
              <LEDGERNAME>Mewar Traders</LEDGERNAME>
              <ISDEEMEDPOSITIVE>No</ISDEEMEDPOSITIVE>
              <AMOUNT>1400.00</AMOUNT>
            </ALLLEDGERENTRIES.LIST>
          </VOUCHER>
        </TALLYMESSAGE>
      </REQUESTDATA>
    </IMPORTDATA>
  </BODY>
</ENVELOPE>
```

8.3 Master Data Ownership & Conflict Resolution Rules

Domain Entity	Authoritative Owner	Conflict Resolution Policy
Customer Sales Orders & Challans	Nagfani Cloud ERP	App is sole authority. Tally receives vouchers via outbox queue.
Warehouse Inventory & Godowns	Nagfani Cloud ERP	Physical & Reserved quantities maintained exclusively in Cloud DB.
Posted Accounting Vouchers	TallyPrime	Once posted, modifications in Tally generate a conflict flag in ERP.
Customer / Supplier Masters	Shared (Two-Way)	Matched on Tally GUID. Master pull imports new ledgers; duplicates go to review queue.
Finished Goods / Raw Item Masters	Shared (Two-Way)	Matched on SKU / Item Name. Tally stock item import updates remote GUID.

9. Real-Time WebSocket Event Dictionary

The backend emits real-time events via Socket.IO after database transactions commit. Frontend clients listen for these events to automatically trigger TanStack Query cache invalidations, ensuring immediate updates across all active terminals without full-page reloads.

Event Name	Triggering Action	Payload Schema	Frontend Cache Invalidation Target
`order:created`	New order confirmed	`{ orderId, orderNo, grandTotal, party }`	`['orders'], ['dashboard'], ['stock_levels']`
`order:status`	Status transitioned (pack/ship/deliver)	`{ orderId, status, previousStatus, order }`	`['orders'], ['order', id], ['delivery_board']`
`stock:changed`	Movements, transfers, adjustments	`{ type, itemId, godownId, level }`	`['stock_levels'], ['stock_movements'], ['stock_reserved']`
`production:created`	Manufacturing batch completed	`{ productionNo, productId, qty, batchNo }`	`['production'], ['stock_levels'], ['dashboard']`
`purchase:created`	Vendor purchase bill inwarded	`{ purchaseNo, supplierId, grandTotal }`	`['purchases'], ['stock_levels'], ['parties']`
`sync:status`	Tally voucher acknowledged/failed	`{ idempotencyKey, status, tallyGuid, error }`	`['sync_overview'], ['sync_log']`
`connector:status`	Desktop gateway heartbeat	`{ tallyReachable, company, version, at }`	`['sync_overview']` (Updates live green pill)
`sync:inbound_masters`	Bulk master pull completed	`{ parties: { created, updated }, items: {...} }`	`['parties'], ['items'], ['sync_overview']`

10. Machine-Readable Quick-Lookup System Index

> **AI System Index:** The following compact JSON manifest provides programmatic summary metadata for automated indexing tools:


```
{
  "system": "Nagfani Foods ERP",
  "version": "1.0.0-classic",
  "architecture": "Distributed 3-Tier (React PWA + Node REST/WS + Windows Tally Gateway)",
  "coreInvariants": {
    "stockFormula": "Available = Physical - Reserved",
    "directStockMutationAllowed": false,
    "gstStateCodeIntra": "08",
    "tallyPort": 9000,
    "outboundSyncProtocol": "Store-and-Forward Idempotent Queue"
  },
  "modules": [
    { "id": "AUTH", "baseRoute": "/api/v1/auth", "models": ["users", "roles"] },
    { "id": "MASTERS", "baseRoute": "/api/v1", "models": ["areas", "godowns", "units"] },
    { "id": "ITEMS", "baseRoute": "/api/v1/items", "models": ["items"] },
    { "id": "PARTIES", "baseRoute": "/api/v1/parties", "models": ["parties", "rate_history"] },
    { "id": "ORDERS", "baseRoute": "/api/v1/orders", "models": ["orders", "counters"] },
    { "id": "STOCK", "baseRoute": "/api/v1/stock", "models": ["stock_movements", "stock_levels", "stock_transfers"] },
    { "id": "PURCHASE", "baseRoute": "/api/v1/purchases", "models": ["purchases"] },
    { "id": "MANUFACTURING", "baseRoute": "/api/v1", "models": ["boms", "production_entries"] },
    { "id": "BILLING", "baseRoute": "/api/v1", "models": ["invoices"] },
    { "id": "PAYMENTS", "baseRoute": "/api/v1/payments", "models": ["payments"] },
    { "id": "SYNC", "baseRoute": "/api/v1/sync", "models": ["sync_outbox", "sync_inbox", "sync_log"] }
  ]
}
```